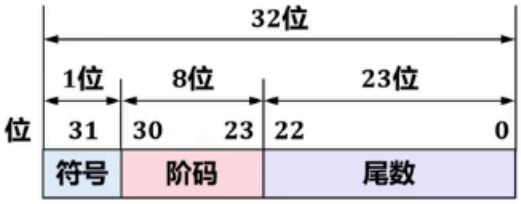
一、32位（单精度）、64位（双精度）浮点数标准

IEEE 754 浮点数标准

■ IEEE 754 标准主要包括两种基本的浮点数格式：

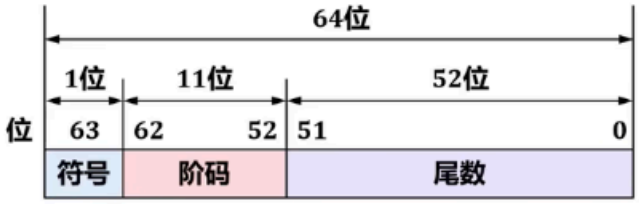
☐ 32位单精度浮点数

对应C语言中的float型。



☐ 64位双精度浮点数

对应C语言中的double型。



其中：

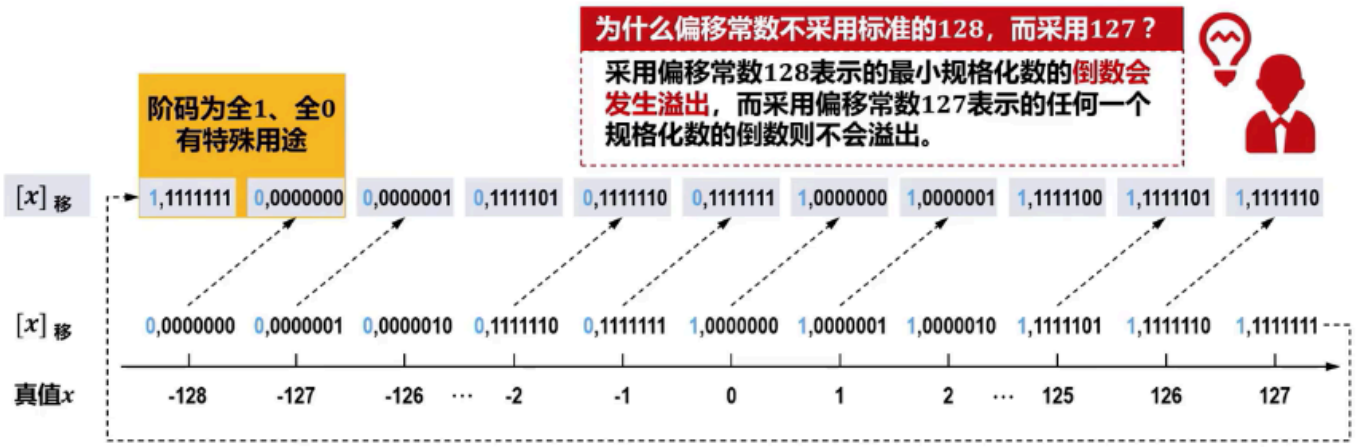
- 符号：取值0表示正数；取值1表示负数。
- 阶码：定点整数，用移码表示。
- 尾数：定点小数，用原码表示。

二、以单精度浮点数说明

2.1 关于阶码的规定

■ 在IEEE 754 浮点数标准中，32位单精度浮点数的8位阶码尽管采用移码表示，但采用偏移常数是$2^7-1=127$，而不是标准移码的$2^7=128$。

$$[x]_{移}=x+(2^7-1) \quad -2^7\le x<2^7$$

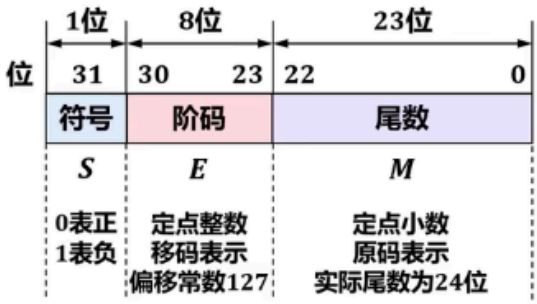


注:

- 关于移码：[[2.6 定点数的移码]][[原码、补码、反码、移码总结]]
- 关于偏移量为何采用127：[[浮点数移码偏移量的规定：IEEE754标准]]

2.2 具体编码规则

■ 下面以32位单精度浮点数为例介绍IEEE 754单精度浮点数标准。



- 符号：取值0表示正数；取值1表示负数。
- 阶码：定点整数，用移码表示，偏置常数$2^7-1=127$。
- 尾数：定点小数，用原码表示。符号位前移到最左侧。
  相邻左侧隐藏一个1，表示数值而不表示符号。
  尾数实际有24位，但不保存隐藏的那个1，只保存23位，节省的比特位可用于提高尾数的精度。
  完整的尾数形式为$1.M$

注:

- 关于1.M：只有规格化的浮点数，才默认隐藏了一个1
- 关于0.M：适用于非规格化浮点数

IEEE 754 单精度（32位）浮点数标准

	数值的分类	符号S	阶码E	尾数M	真值	
机器0	正零	0	0（全0）	0	$+0$	阶码E为全0
机器0	负零	1	0（全0）	0	-0	阶码E为全0
非规格化数	非规格化正数	0	0（全0）	$M\neq 0$	$(-1)^0\times 2^{-126}\times 0.M$	阶码E为全0
非规格化数	非规格化负数	1	0（全0）	$M\neq 0$	$(-1)^1\times 2^{-126}\times 0.M$	阶码E为全0
无穷大	正无穷大	0	255（全1）	0	$+\infty$	阶码E为全1
无穷大	负无穷大	1	255（全1）	0	$-\infty$	阶码E为全1
非数NaN	无定义数（非数）	0或1	255（全1）	$\neq 0$	$NaN(not\ a\ number)$	阶码E为全1
规格化数	规格化正数	0	$1\leq E\leq 254$	M	$(-1)^0\times 2^{E-127}\times 1.M$	阶码E非全0和全1
规格化数	规格化负数	1	$1\leq E\leq 254$	M	$(-1)^1\times 2^{E-127}\times 1.M$	阶码E非全0和全1

- 一般情况下，+0和−0是等效的。
- 非规格化数可用于处理阶码下溢，使得出现比最小规格化数还小的数时程序也能继续进行下去。
- 引入无穷大数可使计算过程出现异常的情况下程序能继续执行，并且可为程序提供错误检测功能。例如非0浮点数除0运算的结果就是无穷大，因此非0浮点数除0不会像整型数除0一样产生严重错误。
- 非数NaN则用于表示$\frac{0}{0}$、$\frac{\infty}{\infty}$、$0\times\infty$、负数的平方根等。部分非数NaN运算结果可能会产生异常。

2.3 单精度、双精度浮点数编码对比

IEEE 754 浮点数标准

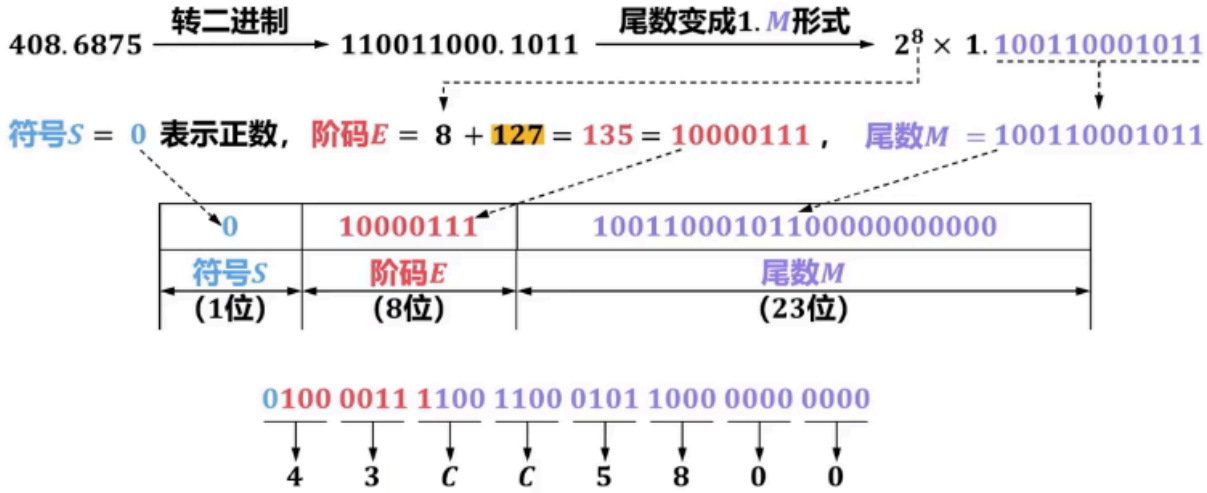
数值的分类	单精度（32位，对应C语言中的float）				双精度（64位，对应C语言中的double）			
	符号S	阶码E	尾数M	真值	符号S	阶码E	尾数M	真值
正零	0	0（全0）	0	$+0$	0	0（全0）	0	$+0$
负零	1	0（全0）	0	-0	1	0（全0）	0	-0
非规格化正数	0	0（全0）	$M\neq 0$	$(-1)^0\times 2^{-126}\times 0.M$	0	0（全0）	$M\neq 0$	$(-1)^0\times 2^{-1022}\times 0.M$
非规格化负数	1	0（全0）	$M\neq 0$	$(-1)^1\times 2^{-126}\times 0.M$	1	0（全0）	$M\neq 0$	$(-1)^1\times 2^{-1022}\times 0.M$
正无穷大	0	255（全1）	0	$+\infty$	0	2047（全1）	0	$+\infty$
负无穷大	1	255（全1）	0	$-\infty$	1	2047（全1）	0	$-\infty$
无定义数（非数）	0或1	255（全1）	$\neq 0$	$NaN(not\ a\ number)$	0或1	2047（全1）	$\neq 0$	$NaN(not\ a\ number)$
规格化正数	0	$1\leq E\leq 254$	M	$(-1)^0\times 2^{E-127}\times 1.M$	0	$1\leq E\leq 2046$	M	$(-1)^0\times 2^{E-1023}\times 1.M$
规格化负数	1	$1\leq E\leq 254$	M	$(-1)^1\times 2^{E-127}\times 1.M$	1	$1\leq E\leq 2046$	M	$(-1)^1\times 2^{E-1023}\times 1.M$

三、例题

1. 例题1

【例题1】将十进制数408.6875转换成IEEE 754单精度浮点数的十六进制机器码。

解析：



注：对于转化过程：符号位是参与的，而且隐藏的1是不需要手动添加的。在底层阶码时，机器会自动识别，做出响应的解码操作

三、关于计算误差

- **十进制小数转换成二进制小数时，可能出现无限循环小数，只能转换到符合精度要求的位数（即所谓的精度缺失）。**
- **IEEE 754浮点数格式位数有限（单精度32位、双精度64位），无法精确表示无限循环小数。**
  - **通常只能采用舍入的方式近似表示，因此会带来数据表示的误差。这种误差会在计算的过程中不断累积放大，可能导致严重后果。**

  【举例】采用双精度浮点数运算

  $$0.1 + 0.1 + 0.1 - 0.3 \neq 0$$

  $$0.7 - 0.2 \neq 0.5$$

  $$3.3 \div 1.1 \neq 3$$

- **综上所述，程序员使用二进制浮点数编程时一定要非常小心，要充分考虑浮点数运算可能带来的计算误差，尽量避免对浮点数进行直接比较，在一些对误差极其敏感的情况下，建议采用十进制浮点数进行运算。**

四、IEEE754关于数据格式的标准

IEEE 754 数据格式规范

标准	C语言数据类型	位宽	基数	阶码位数	阶码偏移常数	尾数位数	最大值量级
半精度	_Float16	16	2	5	15	10	10^5
单精度	float	32	2	8	127	23	10^{38}
扩展单精度		≥ 43	2	≥ 11		≥ 31	
双精度	double	64	2	11	1023	52	10^{308}
扩展双精度	浮点运算器内部	≥ 79	2	≥ 15		≥ 63	
四精度	long double	128	2	15	16383	112	10^{4932}
八精度	暂未支持	256	2	19	262143	236	10^{78913}
Decimal32	_Decimal32	32	10	6	101	20	10^{96}
Decimal64	_Decimal64	64	10	8	398	50	10^{384}
Decimal128	_Decimal128	128	10	12	6176	110	10^{6144}

注：带有背景色的浮点数格式为 IEEE 754 – 2008 新增标准。